Projeto Final - Arquitetura de Software

Relatório Técnico - Arquitetura do EventMaster

1. Contexto e Problema

Durante grandes lançamentos, o sistema cai devido ao alto volume de acessos (latência), a base de dados fica inconsistente e houve tentativas recentes de ataques de injeção de dado.

O sistema possui um grande problema arquitetural, pois não foi projetado para lidar com picos massivos de concorrência e nem para evoluir de forma independente e desacoplada, uma falha no módulo de pagamentos derrubava a visualização de eventos. Funcionalidades como Catálogo, Vendas, Pagamento, Autenticação e Relatórios faziam parte de uma única aplicação com base de dados compartilhada.

Problemas identificados durante grandes lançamentos:

- Alta latência e indisponibilidade
- Contenção no banco de dados
- Escalabilidade limitada (escalabilidade vertical ou replicação total)
- Risco de falhas em cascata
- Vulnerabilidades de segurança

2. Análise da Arquitetura Monolítica Atual

Características: Código único, banco compartilhado, transações distribuídas internamente e deploy único.

Limitação	**Impacto Arquitetural**
Escala uniforme	Todos os módulos escalam juntos
Acoplamento forte	Mudanças tendem a gerar efeitos colaterais
Falha isolada compromete o sistema	Baixa resiliência
Banco único	Gargalo sob alta concorrência

3. Estratégia de Migração

Migração para microsserviços focada em escalabilidade independente, isolamento de falhas, clareza de responsabilidade e independência de deploy. A nova arquitetura do EventMaster utiliza a decomposição funcional para isolar responsabilidades.

Microsserviços: Com escalabilidade horizontal podemos replicar apenas o serviço de Vendas durante um pico de acessos, mantendo o serviço de Catálogo em sua infraestrutura mínima, otimizando custos e recursos.

4. Modelagem do Domínio (DDD)

Para evitar a inconsistência de dados, aplicamos o DDD para definir fronteiras claras de conhecimento e autoridade. Identificamos três **Contextos Delimitados (Bounded Contexts)** principais:

4.1 Contexto de Catálogo de Eventos:

Responsabilidade: Gerenciar a definição, categorização e exposição de informações públicas dos eventos
Entidades: Nome, Data, Local, Descrição.

Comportamento: Alta taxa de leitura. Utiliza estratégias de cache para reduzir a latência durante grandes consultas.

4.2 Contexto de Vendas:

Responsabilidade: Orquestrar o ciclo de vida da aquisição de ingressos, desde a validação de disponibilidade de estoque até a formalização do pedido. No modelo de compra direta, ele é o responsável por garantir que a quantidade vendida não exceda a capacidade do evento.

Entidades: Pedido, Item de Venda, Quantidade, Status do Pedido

Comportamento: Alta criticidade transacional. Este contexto deve ser capaz de lidar com alta concorrência simultânea, assegurando que o estoque seja decrementado de forma atômica e consistente.

4.3 Contexto de Pagamentos

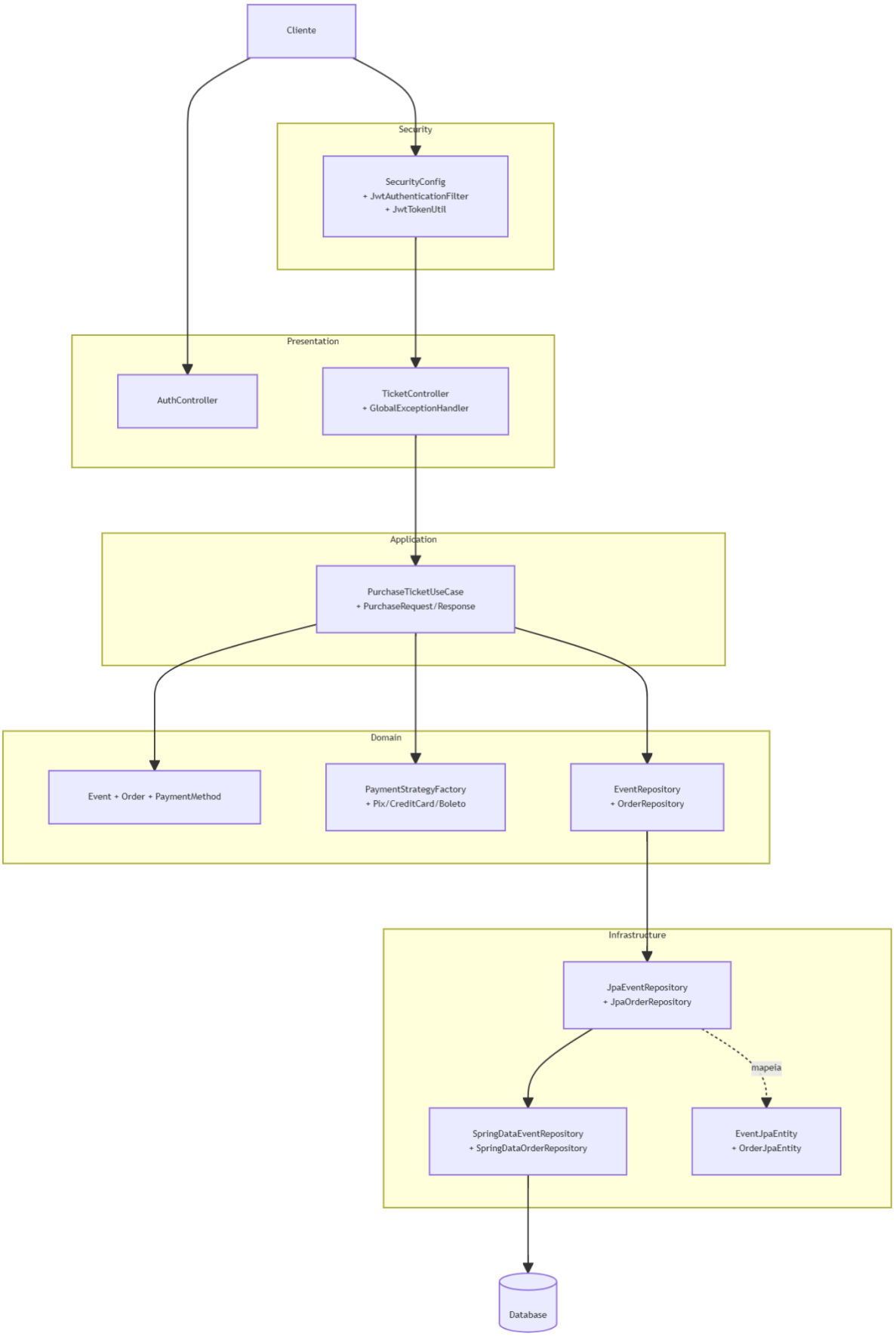
Responsabilidade: Processamento financeiro, traduzir os requisitos financeiros do sistema para integração com entidades de crédito e liquidação. Ele isola a complexidade de taxas, estornos e validações de crédito do restante da lógica de vendas

Entidades: Transação, Método de Pagamento, Gateway, Comprovante

Comportamento: extensibilidade e resiliência, dependência de gateways externos.

5. Arquitetura Proposta

Componente	**Responsabilidade**
API Gateway	Roteamento, autenticação, rate limiting
Serviço de Autenticação	Emissão e validação de tokens
Serviço de Catálogo	Consulta e gestão de eventos
Serviço de Venda	Reserva e confirmação
Serviço de Pagamento	Processamento financeiro


Cliente
Security
SecurityConfig
+ JwtAuthenticationFilter
+ JwtTokenUtil
Presentation
AuthController
TicketController
+ GlobalExceptionHandler
Application
PurchaseTicketUseCase
+ PurchaseRequest/Response
Domain
Event + Order + PaymentMethod
PaymentStrategyFactory
+ Pix/CreditCard/Boleto
EventRepository
+ OrderRepository
Infrastructure
JpaEventRepository
+ JpaOrderRepository
mapeia
SpringDataEventRepository
+ SpringDataOrderRepository
EventJpaEntity
+ OrderJpaEntity
Database

6. Padrões de Resiliência e Consistência

Para um sistema de venda de ingressos, a disponibilidade é tão importante quanto a integridade.

6.1. API Gateway

Atuando como a "porta de entrada" única, o Gateway resolve dois problemas de uma vez:

- **Segurança:** Centraliza a validação do token JWT, impedindo que requisições não autenticadas cheguem aos serviços internos.
- **Abstração:** O cliente não precisa saber o caminho para cada microsserviço, o Gateway faz o roteamento inteligente.

6.2. Circuit Breaker (Disjuntor)

Implementado na comunicação entre o serviço de Vendas e o Gateway de Pagamento externo.

- **Cenário:** Se a operadora de cartão de crédito ficar lenta ou instável, o Circuit Breaker "abre", impedindo que o sistema EventMaster fique "segurando" conexões e gerando latência (timeout). O sistema pode responder rapidamente ao usuário: *"Serviço de pagamento temporariamente indisponível, tente em instantes"*, preservando a saúde do restante da aplicação.

6.3. Padrão SAGA (Coreografia)

Como cada microsserviço possui sua própria base de dados, a consistência transacional é garantida através do padrão SAGA, utilizando uma abordagem de coreografia baseada em eventos assíncronos:

- Fluxo Feliz (Sucesso):
    1. O serviço de Vendas realiza a reserva do ingresso (status: PENDENTE) e publica o evento ReservaRealizada.
    2. O serviço de Pagamento, ao consumir este evento, processa a cobrança junto ao gateway financeiro.
    3. Após a aprovação, o serviço de Pagamento publica o evento PagamentoConfirmado.
    4. O serviço de Vendas consome este evento e atualiza o status do pedido para CONFIRMADO.
- Fluxo de Compensação (Falha):
    1. Caso o pagamento seja negado ou ocorra um erro técnico, o serviço de Pagamento emite o evento PagamentoFalhou.
    2. O serviço de Vendas, que está "ouvindo" este tópico, reage executando a ação compensatória: a reserva é cancelada e o ingresso é devolvido imediatamente ao estoque, garantindo que o sistema não mantenha reservas "presas" e evitando a inconsistência de dados.

7. Processamento de Dados e Performance

A arquitetura de processamento foi desenhada para separar o que exige resposta imediata (crítico para a experiência e segurança) do que pode ser processado de forma assíncrona ou volumétrica.

7.1. Processamento em Stream (Online)

O modelo de *Stream* é aplicado onde a latência é o fator determinante para o sucesso do negócio.

- **Processamento Transacional de Pedidos**: A criação do pedido e a baixa no estoque devem ser processadas em tempo real. Isso garante a consistência imediata do inventário e fornece o *feedback* instantâneo ao cliente.
- **Análise de Segurança e Bloqueio de Bots:** Análise dos logs de acesso e padrões de tráfego conforme eles acontecem. Se o sistema detecta muitas requisições de compra vindo de um único IP ou padrões de comportamento suspeitos, ele dispara um comando de bloqueio imediato no API Gateway.

7.2. Processamento em Batch (Lote / Offline)

O modelo *Batch* é reservado para tarefas que envolvem grandes volumes de dados e que poderiam causar contenção no banco de dados se executadas simultaneamente às vendas.

- **Consolidação Financeira e Payouts**: O cálculo de repasse de valores para os organizadores dos eventos não precisa ocorrer a cada venda. Uma rotina de lote processa todos os pedidos confirmados do dia para gerar a ordem de pagamento única para o produtor.
- **Relatórios de Analytics para Marketing**: Análise profunda de quais regiões ou perfis compraram mais rápido. Como são consultas pesadas que envolvem agregadores (SUM, GROUP BY), são executadas em *batch* para não competir com a CPU do microsserviço de vendas durante o lançamento.

8. Segurança por Design

A segurança é tratada como um requisito arquitetural presente em todas as camadas do sistema para garantir a integridade das transações e a proteção dos dados dos usuários.

8.1 Autenticação e Autorização

O sistema adota um fluxo de autenticação *stateless* baseado em **OAuth 2.0** com a emissão de **JSON Web Tokens (JWT)**.

- **Fluxo de Login**: O usuário realiza a autenticação no endpoint público /auth/login. Após a validação das credenciais, o servidor emite um token JWT assinado digitalmente que contém as permissões do usuário e um tempo de expiração definido.

- **Acesso Protegido**: Para requisições em endpoints sensíveis, como o de compra de ingressos (/api/tickets/purchase), o cliente deve enviar o token no cabeçalho Authorization: Bearer {token}.
- **Validação Centralizada**: **API Gateway** atua como o primeiro filtro, rejeitando requisições com tokens inválidos ou expirados antes mesmo que elas atinjam os serviços internos.

8.2 Arquitetura Zero Trust

Seguindo o princípio de **Zero Trust** (Confiança Zero), a arquitetura assume que a rede interna não é inerentemente segura.

- **Autenticação Inter-Serviços**: Nenhum microsserviço confia implicitamente em outro. Todas as comunicações entre serviços devem ser autenticadas e autorizadas, garantindo que apenas serviços legítimos executem ações no sistema.

8.3 Mitigação de Riscos OWASP Top 10

O sistema foi desenhado para mitigar as principais vulnerabilidades listadas pelo projeto **OWASP**:

- **Broken Access Control**: Mitigado através da validação rigorosa de tokens JWT e escopos de acesso em cada endpoint utilizando o **Spring Security**.
- **Injection**: O uso de **Spring Data JPA** com *Prepared Statements* e a validação rigorosa de entradas de dados impedem ataques de SQL Injection e outros tipos de injeção.
- **Insecure Design**: A aplicação de padrões de resiliência como o **Circuit Breaker** protege o sistema contra falhas de negação de serviço causadas por dependências lentas.
- **Security Misconfiguration**: As configurações de segurança são mantidas de forma imutável através de variáveis de ambiente, evitando falhas de configuração manual no código.
- **Identification and Authentication Failures**: Utilização de tokens JWT com tempo de expiração curto para reduzir a janela de oportunidade em caso de interceptação de tokens.
- **Cryptographic Failures**: Uso obrigatório de protocolos de transporte seguro (**TLS**) para proteger dados sensíveis em tráfego entre o cliente, o gateway e os microsserviços.

9. Qualidade de Código e Testes

O microsserviço de **Venda de Ingressos** foi desenvolvido priorizando a manutenibilidade e a confiabilidade, utilizando princípios de **Arquitetura Limpa**. A lógica de negócio está isolada em uma camada de domínio independente, facilitando a evolução do sistema sem gerar efeitos colaterais.

9.1. Código Limpo e Princípios SOLID

A implementação segue os princípios **SOLID** para garantir que o código seja fácil de compreender e estender:

9.2. Design Pattern (GoF): Strategy

Para lidar com a diversidade de métodos de pagamento exigidos (PIX, CREDIT_CARD, BOLETO), foi aplicado o **Strategy Pattern**.

- **Implementação:** Cada método de pagamento possui sua própria estratégia de processamento encapsulada.
- **Benefício:** Isso elimina o uso de condicionais complexas (`if/else` ou `switch`) na classe principal de vendas, permitindo que o sistema selecione a estratégia correta em tempo de execução com base no input do usuário .

9.3. Estratégia de Testes (Pirâmide de Testes)

A garantia da qualidade do **EventMaster** é sustentada por uma pirâmide de testes que cobre desde a lógica atômica até o comportamento esperado pelo usuário.

- **Testes Unitários (TDD):** Temos aproximadamente 80% de cobertura de linhas de código
- **Teste de Comportamento (BDD):** `CucumberTest`: Implementa cenários em português utilizando Gherkin. Este teste valida o "Caminho Feliz" e fluxos de exceção sob a ótica do usuário final, garantindo que o sistema entregue o valor de negócio esperado (ex: Compra de ingresso com sucesso).

10. Conclusão

A arquitetura proposta para o EventMaster resolve os principais problemas do sistema monolítico ao:

- Permitir escalabilidade independente
- Reduzir acoplamento estrutural
- Isolar falhas
- Aumentar segurança
- Melhorar confiabilidade com testes

Microsserviços introduzem maior complexidade operacional, porém, considerando o cenário de picos massivos de acesso e necessidade de evolução contínua, a abordagem distribuída apresenta melhor alinhamento estratégico com os objetivos do negócio. A abordagem distribuída alinha-se aos objetivos do negócio, priorizando escalabilidade e segurança, fazendo um equilíbrio entre sofisticação arquitetural e viabilidade operacional.